

Chap 1

command line arguments

Date

No.

- Make the input & output filename.ext as variables
- When you run the program from the command line, also pass in the input and output files.

command line arguments

argc: (Argument Count)

An integer that holds the total no. of arguments passed to the program, including the program name itself

argv: (Argument Vector)

An array of `char*` (string pointers) that points to each line argument. Command

`argv[0]` → Always the name / path of the executable program

`argv[1]` → The first user-provided argument "encode10"

`argv[2]` → second "send7.outf10"

Contents of string are entered at the command line.

General command-line syntax

`progname arg1 arg2 ... argn`

- the name of the executable file
- optional arguments, separated by spaces or tabs
- All arguments are treated as character strings

eg. `int main(int argc, char *argv[])` ✖
`{`

`FILE * indata, * outdata;`

`indata = fopen(argv[1], "r");`

`outdata = fopen(argv[2], "w");`

`}`

operations ✖

P1 • Windows: type cmd and press Enter (to open CMD) press Enter

On Windows 7 and above: press Windows+R, type cmd. ✓

- `cd directory_name` (change directory)
- `cd ..` (return to parent directory) multiple times)
- `cd directory_name / directory_name / ...` (change directories)
- `cd ../.. / ...` (return to parent directories multiple times)
- `Tab key` (auto-completion) the current drive)
- `cd \` (changes the current directory to the root directory of ^Y backslash

FALCON

- P2:
- `dir` (display contents of the current directory)
 - `dir directory-name / directory-name /---` (display contents of the specified directory)
 - `dir /a` (display all contents of the current directory, including hidden files)

- P3:
- `cls` (clear the screen)
 - `dir *.*` (display all files in the current directory that match the wildcard)
 - `↑ key` (command history)
 - `Enter a filename` (open/execute the file)
 - `command /?` (view help information) eg. `cd /?`
 - `rename ac1-3.exe Convert.exe`

1) Relative Path Method

All initial files must be in the same directory (including folders, desktop, etc.)

- ① `ac> cd + directory address` (Navigate to the target directory)
- ② `type [target file]` (Prints the contents of the target file in the command prompt)
- ③ `copy [source file full name (including extension)] [initial file]` (automatically generates the target file in the command prompt)

2) Absolute Path Method

- ① Right-click to open and copy the file address (this copied address is an absolute path)
- ② In the command prompt:
`"[source file address]" "[file | address]"`
- ③ `type [target file]` (Prints the content)

Note: if the output file does not exist yet:

- if only the file name (relative path) is written in the program, the operating system will create the output file in the **Current Working Directory (CWD)** when the program runs.

Program.

encode.c

```
#include <stdio.h>
int main (int argc, char* argv[])
{
    FILE* indata, * outdata;
    char this1;
    indata = fopen(argv[1], "r");
    outdata = fopen(argv[2], "w");
    :
    fclose(indata); fclose(outdata);
    return 0;
}
```



convert.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define SIZE 10
int main (int argc, char* argv[])
{
    int digit, base, remainder, i;
    char symbol[SIZE];
    if (argc != 3)
    {
        printf("Wrong Format!");
        exit(1);
    }
    if (strlen(argv[2]) != 7)
    {
        printf("Base Format Error!");
        exit(1);
    }
    if (*argv[2]+6 > '8')
    {
        printf("Base should be less than 9!\n");
        exit(1);
    }
}
```

Turn into num.

```
base = *argv[2]+6 - '0';
digit = atoi(argv[1]); // turn string into integer
if (digit < 0)
    putchar('-');
    digit = -digit;
}
i = 0;
do {
    remainder = digit % base;
    symbol[i] = remainder + '0';
    i++;
    digit = digit / base;
} while (digit > 0);
do {
    i--;
    putchar(symbol[i]);
} while (i != 0);
    putchar('\n');
```